

Week 2 Day 3 Hands-on Lab: Buildable Image Delivery

이 lab은 week2/day3/labs/static-site에 이미 준비된 정적 앱과 Dockerfile을 사용한다. 새 앱을 즉석에서 만들지 않는다. 목표는 제공된 lab을 읽고, build/run/verify 결과값을 해석하고, 실패를 단계별로 분리하는 것이다.

Phase A: lab 파일 읽기

```
cd week2/day3/labs/static-site
pwd
ls -la
sed -n '1,120p' Dockerfile
sed -n '1,120p' .dockerignore
sed -n '1,80p' index.html
```

Expected:

Dockerfile, .dockerignore, index.html, styles.css, README.md가 보인다.
Dockerfile은 nginx:1.27-alpine을 base로 사용한다.
index.html과 styles.css를 /usr/share/nginx/html로 복사한다.
container 내부 port는 80이다.

Phase B: build

```
docker build -t paperclip-static-site:day3 .
docker images paperclip-static-site
```

Expected:

```
naming to docker.io/library/paperclip-static-site:day3
paperclip-static-site  day3  <image-id>  <created>  <size>
```

Phase C: run and verify

```
docker run -d --name paperclip-day3-static -p 18083:80 paperclip-static-site:day3
```

```
docker ps --filter name=paperclip-day3-static
curl -I http://localhost:18083
curl -s http://localhost:18083 | grep "Day 3 Static App"
```

Expected:

```
STATUS Up
0.0.0.0:18083->80/tcp
HTTP/1.1 200 OK
<h1>Day 3 Static App</h1>
```

Phase D: vulnerability scan

```
docker scout version || true
docker scout cves paperclip-static-site:day3 || true
docker scout cves --only-severity critical,high paperclip-static-site:day3 || true
```

Expected:

Scout가 실행되면 CVE summary와 severity를 확인한다.
critical/high가 있으면 safe image 후보로 바로 통과시키지 않고 조치/예외를 기록한다.
Scout가 실행되지 않으면 미설치, 로그인 필요, 네트워크 문제 등 blocker를 기록한다.

Phase E: image evidence

```
docker history paperclip-static-site:day3
docker image inspect paperclip-static-site:day3 --format "{{.Id}} {{.Size}} {{.Architecture}} {{json .RepoTags}}"
```

Expected:

```
COPY index.html ./index.html
COPY styles.css ./styles.css
sha256:<id> <size> amd64 ["paperclip-static-site:day3"]
```

Phase F: cache check

```
printf '\n<!-- day3 rebuild evidence -->\n' >> index.html
docker build -t paperclip-static-site:day3-v2 .
docker images paperclip-static-site
```

Expected:

일부 layer는 CACHED로 재사용된다.
index.html 변경 이후 COPY layer는 다시 실행될 수 있다.
paperclip-static-site:day3-v2 tag가 추가된다.

Phase G: size comparison

```
cd /mnt/d/paperclip/week2/day3/labs/static-site
sed -n '1,120p' Dockerfile.size-compare
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable -t
paperclip-static-site:size-default .
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable-
alpine -t paperclip-static-site:size-alpine .
docker build -f Dockerfile.size-compare --build-arg BASE_IMAGE=nginx:stable-
trixie -t paperclip-static-site:size-trixie .
docker images paperclip-static-site --format "table {{.Repository}} {{.Tag}}
{{.Size}}"
```

Expected:

size-default, size-alpine, size-trixie tag가 보인다.
같은 앱이어도 base image에 따라 size가 다르게 나온다.
학생은 자기 환경의 size 숫자를 기록하고 왜 차이가 나는지 설명한다.

Phase H: failure drill

Missing file build failure:

```
cd /mnt/d/paperclip
cp -r week2/day3/labs/static-site week2/day3/labs/static-site-broken
rm -f week2/day3/labs/static-site-broken/index.html
```

```
cd week2/day3/labs/static-site-broken
docker build -t paperclip-static-site:broken . || true
```

Expected failure output:

```
ERROR ... COPY index.html ./index.html
failed to calculate checksum
"/index.html": not found
```

Hint:

COPY 단계에서 실패했다면 build context 안에 source file이 있는지 본다.

Wrong port verify failure:

```
cd /mnt/d/paperclip
docker run -d --name paperclip-day3-static-wrong -p 18084:8080 paperclip-
static-site:day3 || true
curl -I http://localhost:18084 || true
docker ps -a --filter name=paperclip-day3-static-wrong
```

Expected failure output:

```
curl: (52) Empty reply from server
# 또는 curl: (7) Failed to connect ...
PORTS 0.0.0.0:18084->8080/tcp
```

Hint:

Dockerfile은 EXPOSE 80인데 run 명령은 container 8080으로 publish했다.

Wrong CMD run failure:

```
cd /mnt/d/paperclip/week2/day3/labs/static-site
awk '{ if ($1 == "CMD") print "CMD [\"nginx-bad-command\"]"; else print $0 }'
Dockerfile > Dockerfile.bad-cmd
docker build -f Dockerfile.bad-cmd -t paperclip-static-site:bad-cmd .
docker run -d --name paperclip-day3-bad-cmd paperclip-static-site:bad-cmd ||
true
docker ps -a --filter name=paperclip-day3-bad-cmd
docker logs paperclip-day3-bad-cmd --tail 30 || true
```

Expected failure output:

```
STATUS Exited (127)
exec: "nginx-bad-command": executable file not found in $PATH
```

Hint:

container가 바로 Exited 되면 docker logs와 image Config.Cmd를 확인한다.

Bloated context hygiene failure:

```
cd /mnt/d/paperclip/week2/day3/labs/static-site
mkdir -p node_modules __pycache__ dist build coverage tmp
printf "DO_NOT_COMMIT_TOKEN=example" > .env
printf "large dependency placeholder" > node_modules/example.txt
printf "compiled output" > dist/app.bundle.js
find . -maxdepth 2 -type f | sort
du -sh .
sed -n '1,160p' .dockerignore
rm -rf .env node_modules __pycache__ dist build coverage tmp Dockerfile.bad-
cmd
```

Expected output:

```
.env
node_modules/example.txt
dist/app.bundle.js
```

Hint:

build가 성공해도 .env, node_modules, dist가 context에 섞이면 보안/속도/비용 위험이다.

RCA note:

```
Symptom:
Stage: build / run / verify / hygiene
Expected result:
Actual result:
Output hint:
First evidence command:
Cause:
```

Fix:
Recheck:

Phase I: tag and registry gate

```
docker tag paperclip-static-site:day3 paperclip-static-site:day3-reviewed
docker tag paperclip-static-site:day3 paperclip-static-site:v1.0.0
docker tag paperclip-static-site:day3 paperclip-static-site:staging
docker images paperclip-static-site
docker image inspect paperclip-static-site:day3-reviewed --format "{{json
.RepoTags}} {{json .RepoDigests}}"
```

Expected:

paperclip-static-site:day3-reviewed가 추가된다.
paperclip-static-site:v1.0.0은 웹앱 릴리즈 버전 예시다.
paperclip-static-site:staging은 환경 tag 예시다.
local image에서는 RepoDigests가 비어 있을 수 있다.
registry에 push/pull된 image는 digest 확인이 가능하다.

Docker Hub push는 선택이다. push 전에는 다음 gate를 통과해야 한다.

- repository 공개 범위를 설명할 수 있다.
- image/context에 secret이 들어가지 않았음을 확인했다.
- tag 이름이 앱 버전, 환경, 빌드 출처, 검수 상태 중 무엇을 표현하는지 설명할 수 있다.
- version tag는 웹 애플리케이션의 실제 릴리즈 버전과 맞춘다.
- credential/token이 README, screenshot, terminal output에 남지 않는다.

Weekend Challenge: 3-tier runtime evidence

Day 3 기본 lab을 끝낸 뒤에는 session-09-challenge.md를 기준으로 주말 챌린지를 진행한다.

```
cd /mnt/d/paperclip/week2/day3
sed -n '1,260p' session-09-challenge.md
sed -n '1,180p' labs/weekend-3tier-challenge/SUBMISSION.md
```

목표는 frontend, backend, database를 각각 다른 책임으로 실행하고 다음 증거를 제출 표에 남기는 것이다. Frontend는 index.html과 styles.css, backend는 Node.js hello world, database는 PostgreSQL 16 + named volume이면 충분하다.

Evidence	최소 통과 기준
Architecture	frontend, backend, database container와 network 역할을 표로 설명한다.
Network	frontend -> backend 성공, backend -> db 성공, frontend -> db 실패를 command output으로 남긴다.
Volume	paperclip-weekend-pgdata named volume과 재시작 후 Skipping initialization 신호를 확인한다.
Logs	frontend/backend/db 각각 docker logs --tail 핵심 line을 남긴다.
Troubleshooting	502, DB 연결 실패, stale volume, build bloating 중 하나를 RCA 형식으로 정리한다.
Optimization	frontend/backend base image별 최초 build seconds와 image size를 표로 비교한다.
Tag or upload	push는 선택이지만 tag 또는 registry URL을 남긴다.

이 챌린지는 Docker Compose 없이 일부러 docker network, docker run, docker exec, docker logs, named volume을 직접 사용한다. 다음 주 Kubernetes와 AWS/Terraform에서는 같은 구조를 더 선언적인 방식으로 다시 표현한다.

Cleanup

```
docker stop paperclip-day3-static paperclip-day3-static-wrong paperclip-day3-bad-cmd || true
docker rm paperclip-day3-static paperclip-day3-static-wrong paperclip-day3-bad-cmd || true
rm -rf /mnt/d/paperclip/week2/day3/labs/static-site-broken
# 필요할 때만 image 삭제
# docker image rm paperclip-static-site:day3 paperclip-static-site:day3-v2
paperclip-static-site:day3-reviewed paperclip-static-site:v1.0.0 paperclip-static-site:staging
paperclip-static-site:size-default paperclip-static-site:size-alpine paperclip-static-site:size-trixie
paperclip-static-site:broken paperclip-static-site:broken-fixed paperclip-static-site:bad-cmd
```